Due: Thursday, Oct 22, 2026 11:59pm

Submit Q1 using the Google Form at (clickable link): https://forms.gle/m7j7XxV2khfkm39QA

Submit Q2 on GL.

---

1 Predictive Parsing

Using the grammar and parse table below, show every step of a predictive parser processing the given token stream.

1	S	⟶	E $
2	E	⟶	T Ttl
3	Ttl	⟶	+ T Ttl
4	Ttl	⟶	ε
5	T	⟶	F Ftl
6	Ftl	⟶	* F Ftl
7	Ftl	⟶	ε
8	F	⟶	(E)
9	F	⟶	id

Top-of-stack nonterminal	Current input token					
	id	+	*	(	)	$
S	1	–	–	1	–	–
E	2	–	–	2	–	–
Ttl	–	3	–	–	4	4
T	5	–	–	5	–	–
Ftl	–	7	6	–	7	7
F	9	–	–	8	–	–

Parse Stack	Input Stream	Comment
S	(id id * id) $	initial contents
E $	(id id * id) $	use S ⟶ E $
T Ttl $	(id id * id) $	use E ⟶ T Ttl

2 Lexical Analysis using Haskell & PCRE Regex.

Your assignment is to modify a Haskell function `lexer` that scans the text of a program and converts it into a token stream. The token stream is a Haskell list. Each item of this list has the `Token` data type, defined by:

```
data EType = NADA | EDllr | ESclr | EBool
             deriving (Show, Read, Eq)

data IdKind = RIEN | LocalVarId | GlobalVarId | FunPrcId
            deriving (Show, Read, Eq)

data Token

   -- literal constants
   = TkBool   Bool       -- boolean true or false
   | TkDllr   Int        -- dollar literal
   | TkSclr   Int        -- scalar literal
   | TkStrg   String     -- literal string

   -- identifiers
   | TkId IdKind EType String

   -- more
   | TkKW    String     -- keyword
   | TkOp    String     -- unary or binary operator
   | TkComp  String     -- comparison operator
   | TkPunc  String     -- punctuation

   -- special cases
   | TkError String     -- error found with message
   | TkEOF              -- end of file

   deriving (Show, Eq, Read)
```

The `lexer` function has type:

```
lexer :: String -> [ Token ]
```

The definition of the `Token` type, the example `lexer` function and a main program are in the files `Token.hs`, `LangTypes.hs`, `basiclex.hs` and `DoLex.hs`. These files along with some test files are available on GL at:

```
/afs/umbc.edu/users/c/h/chang/pub/cs331stuff/hw06/
```

First, look through the `basiclex.hs` file and make sure you understand how to use the regex matching operator $=\sim$. Also, make sure that you understand how pattern guards and recursion work in this example. This was explained during lecture on Thursday October 15. Make sure you understand why we have to use `\A` instead of the usual ^ in our regex patterns.

The test files are `test1.prg`, ..., `test9.prg`. The token streams for these test files are in `test3.tok`, ..., `test9.tok`. You can check your output with:

```
linux3% ./a.out < test9.prg > mytest9.tok
linux3% diff test9.tok mytest9.tok
```

Note that until you finish all of the tasks below, your output will differ from the standard results.

Important Stipulation: You are not allowed to use lookaheads in your regular expressions. You should not have any use of `(?=`, `(?!`, `(?<=` or `(?<!"` in any of your expressions. You may use `(?:` for non-capture grouping, but it is not needed.

Tasks:

1. Rename the `basiclex.hs` file `Lexer.hs`. Put your name on top, but leave the module name as `Lexer`. The top of your file should look like:

   ```
   module Lexer where
   -- Ncuti Gatwa (doc15)
   ```

2. Make sure your files compile and run, before you make any more changes.

   ```
   linux3% ghc --make -o a.out DoLex.hs
   [1 of 4] Compiling LangTypes        ( LangTypes.hs, LangTypes.o )
   [2 of 4] Compiling Token            ( Token.hs, Token.o )
   [3 of 4] Compiling Lexer            ( Lexer.hs, Lexer.o )
   [4 of 4] Compiling Main             ( DoLex.hs, DoLex.o )
   Linking a.out ...
   linux3%
   ```

   You should be able to run the `a.out` program and test it on a simple example:

   ```
   linux3% cat test1.prg
   : 3x >R 3 R@ * Rdrop ;
   linux3% ./a.out < test1.prg
   [TkPunc ":",TkId LocalVarId ESclr "3x",TkId LocalVarId ESclr ">R",
   TkSclr 3,TkId LocalVarId ESclr "R@",TkId LocalVarId ESclr "*",
   TkId LocalVarId ESclr "Rdrop",TkPunc ";",TkEOF]
   linux3%

   linux3% cat test2.prg
   println "Hello World" ;
   linux3% ./a.out < test2.prg
   [TkId LocalVarId ESclr "println",TkStrg "Hello World",TkPunc ";",TkEOF]
   linux3%
   ```

3. The current `lexer` function recognizes `:` and `;` as punctuation in a program and assumes that all lexical elements are separated by white space. This might be appropriate for some languages, but we want a more sophisticated programming language than that.

   Modify `lexer` so it scans for parentheses `( )`, comma `,` and semicolon `;` as punctuation. (Note: do remove `:` from the list.)

   The punctuation matched should be saved as a `String` value in the `TkPunc`.

4. Next, modify `lexer` so it identifies single-character operators: `< > * / + - ~` You should use a separate guard for this case.

   Recall that `-` has special meaning in a character class and does need to be escaped. It needs to be escaped with a backslash `\`. However, Haskell uses backslashes in a string for control characters (e.g., `\n` for newline). So, you need to use two backslashes in a Haskell string in order to get one stored in the regex pattern.

   These should be added to the token stream as a `TkOp` item. The operator matched should be saved as a `String` value in the `TkOp`.

5. Next, modify `lexer` so it identifies two-character operators: `:= <= == != >= && and ||`.

   You should use a separate guard for this case. Think about where this guard should go. Before the single-character operators? or after? What about punctuation?

   These should be added to the token stream as a `TkOp` item. The operator matched should be saved as a `String` value in the `TkOp`.

   Note: The `||` operator needs special attention since `|` has special meaning in PCRE and must be escaped.

   Another Note: You should test the `lexer` function regularly for debugging. Just remember to enter Control-D to terminate your input.

```
linux3% ghc --make -o a.out DoLex.hs
[3 of 4] Compiling Lexer            ( Lexer.hs, Lexer.o )
Linking a.out ...
linux3% ./a.out
3 + 4 * 5
[TkSclr 3,TkOp "+",TkSclr 4,TkOp "*",TkSclr 5,TkEOF]
linux3%
```

6. Note: next, remove the `\\s` from the regex for numbers.

   Then, modify the identifier pattern in `lexer`. The current pattern allows identifiers that start with a digit, but in our programming language, we want identifiers to start with an upper or lower case letter and then followed optionally by any number of letters or digits.

   Our programming language has 3 types of values: dollar amounts, scalar values and boolean values. To aid in code generation later, boolean identifiers will have a `?` suffix and dollar identifiers will have a `$` suffix. Scalar identifiers have no suffix. Furthermore, global varialbes are prefixed by `&`, functions are prefixed by and procedures are prefixed by `_`.

   Identifiers should be added to the token stream as a `TkId` which was definded as:

```
TkId IdKind EType String
```

The `IdKind` field holds the kind of the identifier and the `EType` field stores the type of the identifier. The `String` field should hold the name of the identifer without any prefixes or suffixes. For example,

```
linux3% ./a.out
abc abc$ abc?
[TkId LocalVarId ESclr "abc",
 TkId LocalVarId EDllr "abc",
 TkId LocalVarId EBool "abc",TkEOF]
linux3%

linux3% ./a.out
&abc &abc$ &abc?
[TkId GlobalVarId ESclr "abc",
 TkId GlobalVarId EDllr "abc",
 TkId GlobalVarId EBool "abc",TkEOF]
linux3%

linux3% ./a.out
@abc @abc$ @abc? _abc
[TkId FunPrcId ESclr "abc",
 TkId FunPrcId EDllr "abc",
 TkId FunPrcId EBool "abc",
 TkId FunPrcId NADA "abc",TkEOF]
linux3%
```

Hint: make some helper functions to assist in classifying identifiers.

7. Modify lexer to recognize scalar literals such as `313`, `3.1415`, `3.10` or `0.3`. Note that the fractional part is optional. Furthermore, we will store only 4 digits of precision to the right of the decimal point. The source program can have additional digits, but extra digits will be truncated (not rounded). To keep code generation simple, we will store scalar literals as the actual value times 10000. For example,

```
linux3% ./a.out
313 3.14159 3.10 0.3
[TkSclr 3130000,TkSclr 31415,TkSclr 31000,TkSclr 3000,TkEOF]
linux3%
```

Each scalar literal should be added to the token stream as a `TkSclr` with the associated value $\times 10000$. In Haskell, you can convert a `String` value to a `Int` using

```
read str :: Int
```

8. Modify lexer to recognize dollar amount literals such as `$1,234,567.89`, `$1234567` and `$1234.567`. Note that the `$` prefix is required. Also, if commas are used then they must be used every third digit. Thus, `$1234,567` would be a syntax error. Similar to scalar literals, we will keep only 2 digits of precision. Additional digits are allowed, but will be truncated. In the `TkDllr` tokens, we will store the values $\times 100$ (i.e., as cents). For example,

```
linux3% ./a.out
$1,234,567.89 $1234567 $1234.567
[TkDllr 123456789,TkDllr 123456700,TkDllr 123456,TkEOF]
linux3%
```

Hint: it is much simpler to handle dollar literals with two pattern guards: the first one handles dollar literals with commas, the second without.

9. Next, modify lexer to ignore comments. We will use C++ style one-line comments. One-line comments start with `//` and continue until the end of the line. For example,

```
linux3% ./a.out
// foo bar galore
[TkEOF]
linux3%
```

10. Now, modify lexer to ignore multi-line comments. We will use braces { } for multiline comments:

```
linux3% ./a.out
{ this is a comment
  still a comment
  last comment line }
[TkEOF]
linux3%
```

11. Finally, add to `lexer` the ability to recognize keywords and the Boolean constants `true` and `false`. We want the following strings to be keywords in our programming language:

```
begin     do      else    elsif   end     extern
false     for     global  if      loop    print
println   return  then    true    until   while
```

These should be added to the token stream as a `TkKW` item. The keyword matched should be saved as a `String` value in the `TkKW`. The Boolean constants on the other hand, should be stored in a `TkBool` token. For example:

```
linux3% ./a.out
if true else false while for return
[TkKW "if",TkBool True,TkKW "else",TkBool False,
TkKW "while",TkKW "for",TkKW "return",TkEOF]
linux3%
```

Note: Think about how scanning for identifiers and keywords interact. In particular, you can have an identifier that has a keyword as a prefix (e.g., `dock`, `forest`, `printer`).

12. Finally, submit just your `Lexer.hs` file on GL as usual.